



AHMED HUSSAIN

547096



DSA

LAB NO. 01



National University of Sciences and Technology (NUST)
School of Electrical Engineering and Computer Science

Department of Computing

CS: Data Structures and Algorithms

Class: BSCS-15D / BSCS-15E

Lab 01: Introduction to Version Control (Git) and Simple Test Cases

CLO 3: Practice programs using current IDEs while testing and documenting programs according to accepted standards.

CLO 4: Demonstrate professional practice through meaningful commits, attribution, and academic integrity.

Date: 8th September 2026

Time: 2 PM – 5 PM

Instructor: Sara Tariq Sheikh

Lab Engineer: Urooj Akmal

Lab 1: Arrays, Basic OOP and Git



Introduction

This lab revisits small C++ programs before moving to more complex data structures. You will practise array indices and loops, create a class and objects, check outputs manually, and save your work with Git. Work through one task at a time.

Objectives

- Declare an integer array and identify its valid indices.
- Read, update and display array elements using a loop.
- Calculate a total and check it using simple test cases.
- Distinguish a class from an object and use data members and a member function.
- Create a local Git repository and save two meaningful commits.

Tools/Software Requirements

A C++ compiler/IDE and Git with a terminal or Git Bash. Use the IDE to compile and run each source file separately. Your instructor will help with Git setup.

Arrays

- An array stores a fixed number of elements of the same type. Each element has an index (position number). C++ array indices start at 0.

```
int values[3] = {5, 10, 15};  
cout << values[0];  
values[1] = 12;
```

Index	0	1	2
Original value	5	10	15
After the update	5	12	15

For an array of size 3, valid indices are 0, 1 and 2. Accessing values[3] is outside the array and is invalid. It is not guaranteed to produce an error message.

```
for (int i = 0; i < 3; i++) {  
    cout << values[i] << " ";  
}
```



}

The loop starts at 0, continues while $i < 3$, and increases i by 1 after each iteration. It displays: 5 12 15.

Object-Oriented Programming (OOP)

A class is a user-defined type that groups related data and functions. An object is an instance of that class. Each object has its own values for the ordinary data members used here.

```
class Box {  
public:  
    int length;  
    void show() {  
        cout << length << "\n";  
    }  
};  
// Inside main():  
Box b;  
b.length = 4;  
b.show();
```

Box is the class; b is an object; length is a data member; show() is a member function. The dot accesses an object's members. public allows access from main(). void means the function returns no value. A class definition ends with a semicolon.

These are fragments: place `#include <iostream>` and `using namespace std;` above the class, and place object creation and use inside main().

Lab Tasks:

Github Link: <https://github.com/ahmedimcb12-bot/lab01-dsa.git>

Task 1:

Create an array named numbers containing 2, 4, 6, 8 and 10. Change the third element to 7. Use a for loop to display all five values, separated by spaces. Predict the output before running your program.



```
main.cpp
1  #include<iostream>
2  using namespace std;
3  int main(){
4      int numbers[5]={2,4,6,8,10};
5      for(int i =0;i<5;i++){
6          if(i==3){
7              numbers[i]=7;
8          }
9          cout<<numbers[i]<<" ";
10     }
11 }
12 }
```

input

2 4 6 7 10

...Program finished with exit code 0
Press ENTER to exit console.

Task 2:

Declare `int numbers[5]`; and initialise `int total = 0`; Use one loop to read five integers into the array and a second loop to add them to total. Display the total. Assume the user enters exactly five valid integers whose sum fits in `int`.

Test input	Expected total	Actual total
1 2 3 4 5	Complete before running	15
0 0 0 0 0	Complete before running	0



-2 4 -1 0 3

Complete before running

4

For the first test, record i , $\text{numbers}[i]$ and total after each addition. If a result differs from your prediction, check the loop bounds and the initial value of total .

```
main.cpp
1  #include<iostream>
2  using namespace std;
3  int main(){
4      int numbers[5];
5      int total=0;
6      cout<<"Enter the numbers "<<endl;
7      for(int i=0;i<5;i++){
8          cin>>numbers[i];
9      }
10     for(int j=0;j<5;j++){
11         total+=numbers[j];
12     }
13     cout<<"The total value is "<<total;
14 }
```

input

```
Enter the numbers
1
2
3
4
5
The total value is 15
```



National University of Sciences and Technology (NUST)

School of Electrical Engineering and Computer Science

```
main.cpp
1  #include<iostream>
2  using namespace std;
3  int main(){
4      int numbers[5];
5      int total=0;
6      cout<<"Enter the numbers "<<endl;
7      for(int i=0;i<5;i++){
8          cin>>numbers[i];
9      }
10     for(int j=0;j<5;j++){
11         total+=numbers[j];
12     }
13     cout<<"The total value is "<<total;
14 }
```

input

```
Enter the numbers
0
0
0
0
0
The total value is 0
...Program finished with exit code 0
Press ENTER to exit console.
```



```
main.cpp
1  #include<iostream>
2  using namespace std;
3  int main(){
4      int numbers[5];
5      int total=0;
6      cout<<"Enter the numbers "<<endl;
7      for(int i=0;i<5;i++){
8          cin>>numbers[i];
9      }
10     for(int j=0;j<5;j++){
11         total+=numbers[j];
12     }
13     cout<<"The total value is "<<total;
14 }
```

input

```
Enter the numbers
-2
4
-1
0
3
The total value is 4
...Program finished with exit code 0
```

Task 3:

Define a class named Student with two public integer data members, rollNumber and marks. Add a public member function void display() that prints both values with labels on separate lines. Define the class above main().

Inside main(), create objects s1 and s2. Assign and display the following values using the dot operator and display():

Object	rollNumber	marks
s1	1	75
s2	2	90



National University of Sciences and Technology (NUST) School of Electrical Engineering and Computer Science

Write the expected output before running. Then change only s1.marks to 80 and display both objects again. Predict whether s2.marks changes, and explain the result. Keep this experiment in your final program.

```
1 #include <iostream>
2 using namespace std;
3
4 class Student {
5 public:
6     int rollNumber;
7     int marks;
```



```
9 void display() {  
10     cout << "Roll Number: " << rollNumber << "\n";  
11     cout << "Marks: " << marks << "\n";  
12 }  
13 };  
14  
15 int main() {  
16     Student s1;  
17     s1.rollNumber = 1;  
18     s1.marks = 75;  
19  
20     Student s2;  
21     s2.rollNumber = 2;  
22     s2.marks = 90;  
23  
24     cout << "--- Initial Values ---\n";  
25     cout << "Student 1:\n";  
26     s1.display();  
27     cout << "Student 2:\n";  
28     s2.display();  
29  
30     // Modify only s1  
31     s1.marks = 80;  
32  
33     cout << "\n--- After modifying s1.marks ---\n";  
34     cout << "Student 1:\n";  
35     s1.display();  
36     cout << "Student 2:\n";  
37     s2.display();  
38  
39     return 0;  
40 }
```

```
--- After modifying s1.marks ---  
Student 1:  
Roll Number: 1  
Marks: 80  
Student 2:  
Roll Number: 2  
Marks: 90
```

```
--- Initial Values ---  
Student 1:  
Roll Number: 1  
Marks: 75  
Student 2:  
Roll Number: 2  
Marks: 90
```

Task 4:



National University of Sciences and Technology (NUST)

School of Electrical Engineering and Computer Science

Read 8 integers into an array. Find and display the largest and smallest values and their indices. If a value occurs more than once, report its first occurrence.



main.cpp

```
1  #include<iostream>
2  #include<climits>
3  using namespace std;
4
5  int main(){
6      int n[8];
7      cout<<"Enter the numbers\n";
8      for(int i=0;i<8;i++){
9          cin>>n[i];
10     }
11     int a=INT_MIN;
12     int p;
13     for(int j=0;j<8;j++){
14         if(a<n[j]){
15             a=n[j];
16             p=j;
17         }
18     }
19     cout<<"The Largest number is "<<a<<" at "<<p<<endl;
20     int b=INT_MAX;
21     int q;
22     for(int k=0;k<8;k++){
23         if(b>n[k]){
24             b=n[k];
25             q=k;
26         }
27     }
28     cout<<"The smallest number is "<<b<<" at "<<q<<endl;
29
30     for(int l = 0; l < 8; l++){
31
32         bool alreadyPrinted = false;
33
34
35         for(int p = 0; p < l; p++){
36             if(n[l] == n[p]){
37                 alreadyPrinted = true;
38                 break;
39             }
40         }
41
42         if(alreadyPrinted)
43             continue;
44
45
46         for(int j = l + 1; j < 8; j++){
47             if(n[l] == n[j]){
48                 cout << "The repeated number is " << n[l] << endl;
49                 break;
50             }
51         }
52     }
53 }
```



National University of Sciences and Technology (NUST) School of Electrical Engineering and Computer Science

```
Enter the numbers
1
2
3
4
5
6
4
3
The Largest number is 6 at 5
The smallest number is 1 at 0
The repeated number is 3
The repeated number is 4
```

Task 5:

Read 8 integers into an array. Find and display the largest and smallest values and their indices. If a value occurs more than once, report its first occurrence.



main.cpp

```
1  #include<iostream>
2  #include<climits>
3  using namespace std;
4
5  int main(){
6      int n[8];
7      cout<<"Enter the numbers\n";
8      for(int i=0;i<8;i++){
9          cin>>n[i];
10     }
11     int a=INT_MIN;
12     int p;
13     for(int j=0;j<8;j++){
14         if(a<n[j]){
15             a=n[j];
16             p=j;
17         }
18     }
19     cout<<"The Largest number is "<<a<<" at "<<p<<endl;
20     int b=INT_MAX;
21     int q;
22     for(int k=0;k<8;k++){
23         if(b>n[k]){
24             b=n[k];
25             q=k;
26         }
27     }
28     cout<<"The smallest number is "<<b<<" at "<<q<<endl;
29
30     for(int l = 0; l < 8; l++){
31
32         bool alreadyPrinted = false;
33
34
35         for(int p = 0; p < l; p++){
36             if(n[l] == n[p]){
37                 alreadyPrinted = true;
38                 break;
39             }
40         }
41
42         if(alreadyPrinted)
43             continue;
44
45
46         for(int j = l + 1; j < 8; j++){
47             if(n[l] == n[j]){
48                 cout << "The repeated number is " << n[l] << endl;
49                 break;
50             }
51         }
52     }
53 }
```



National University of Sciences and Technology (NUST) School of Electrical Engineering and Computer Science

```
Enter the numbers
1
2
3
4
5
6
4
3
The Largest number is 6 at 5
The smallest number is 1 at 0
The repeated number is 3
The repeated number is 4
```

Task 6:

Read 6 integers into an array. Reverse the elements **without using another array**, then display the updated array.

Example: {3, 7, 2, 9, 4, 1} becomes {1, 4, 9, 2, 7, 3}.



```
main.cpp
1  #include<iostream>
2  using namespace std;
3  int main(){
4      int n[6]={3,7,2,9,4,1};
5      int i=0;
6      int j=5;
7      int b;
8      while(i<j){
9          b=n[i];
10         n[i]=n[j];
11         n[j]=b;
12         i++;
13         j--;
14     }
15     for(int i=0;i<6;i++){
16         cout<<n[i]<<" ";
17     }
18 }
```

1 4 9 2 7 3

...Program finished with exit code 0
Press ENTER to exit console.

Task 7:

Read 10 integers into an array. Move the first occurrence of each distinct value into the beginning of the same array, preserving the original order. Display these unique values and their count. Do not use another array.

Example: {4, 2, 4, 7, 2, 9, 7, 1, 9, 1} produces {4, 2, 7, 9, 1}, with a count of 5.

The array's physical capacity remains 10; only the first 5 positions represent the result.



National University of Sciences and Technology (NUST)
School of Electrical Engineering and Computer Science

main.cpp

```
1  #include <iostream>
2  using namespace std;
3
4  int main() {
5      int a[10];
6      int count = 0;
7
8      cout << "Enter 10 integers:\n";
9
10     for (int i = 0; i < 10; i++) {
11         cin >> a[i];
12     }
13
14     for (int i = 0; i < 10; i++) {
15
16         bool duplicate = false;
17
18         for (int j = 0; j < i; j++) {
19             if (a[i] == a[j]) {
20                 duplicate = true;
21                 break;
22             }
23         }
24
25         if (!duplicate) {
26             a[count] = a[i];
27             count++;
28         }
29     }
30
31     cout << "Unique values: ";
32
33     for (int i = 0; i < count; i++) {
34         cout << a[i] << " ";
35     }
36
37     cout << "\nCount = " << count << endl;
38
39     return 0;
40 }
```



```
Enter 10 integers:
1
2
3
4
5
3
2
1
2
4
Unique values: 1 2 3 4 5
Count = 5
```

Task 8:

Create a folder named dsa-lab01 and place your tasks inside it. Open a terminal in that folder. Git setup is to be demonstrated by the lab engineer.

- git init
 - git status
 - git add task2.cpp task3.cpp task4.cpp
 - git commit -m "Add array and Student class exercises"
 - Create README.md with your name, registration number, one sentence about each program, and the three Task 3 test results. Then run:
 - git add README.md
 - git commit -m "Document programs and test results"
 - git status
 - git log --oneline
 - git add stages selected files. git commit records the staged version locally. git status shows file changes; git log shows commits. A commit does not upload files to GitHub. Stage only the named source and README files, not executables.
-
- If Git asks for an identity, the instructor will help you run git config user.name "Your Name" and git config user.email "your-email@example.com" with your own details. These settings apply to this repository.

Deliverables



National University of Sciences and Technology (NUST) School of Electrical Engineering and Computer Science

- Submit the .cpp files, README.md and a short PDF report on LMS (or the repository link if permitted). The report should contain your name and registration number.
- Academic integrity: Discuss ideas, but type, run and explain your own programs. Acknowledge any external code or explanations used.

Lab Rubrics

Total: 10 marks. A short viva may be used to check individual understanding.

Criterion	Marks	Evidence expected
Array concepts	2	Task 1 answers and Task 2 explanation.
Array programs	3	Correct update/display (1), input/total (1), test table and trace (1).
Basic OOP	3	Class and members (1), two objects and output (1), explanations (1).
Git practice	1	Two meaningful commits and log evidence.
Submission and explanation	1	Readable code, complete files and individual understanding.

Reference

- Goodrich, M. T., Tamassia, R., & Mount, D. M. (2011). Data Structures and Algorithms in C++ (2nd ed.). Wiley. Consult the introductory C++ and object-oriented programming material.